



Apache Spark

Лекция 3





Я Александр
Бадьин

Ведущий разработчик
Inference-платформы

11:25



План занятия

Недостатки Hadoop

Основные концепции Spark

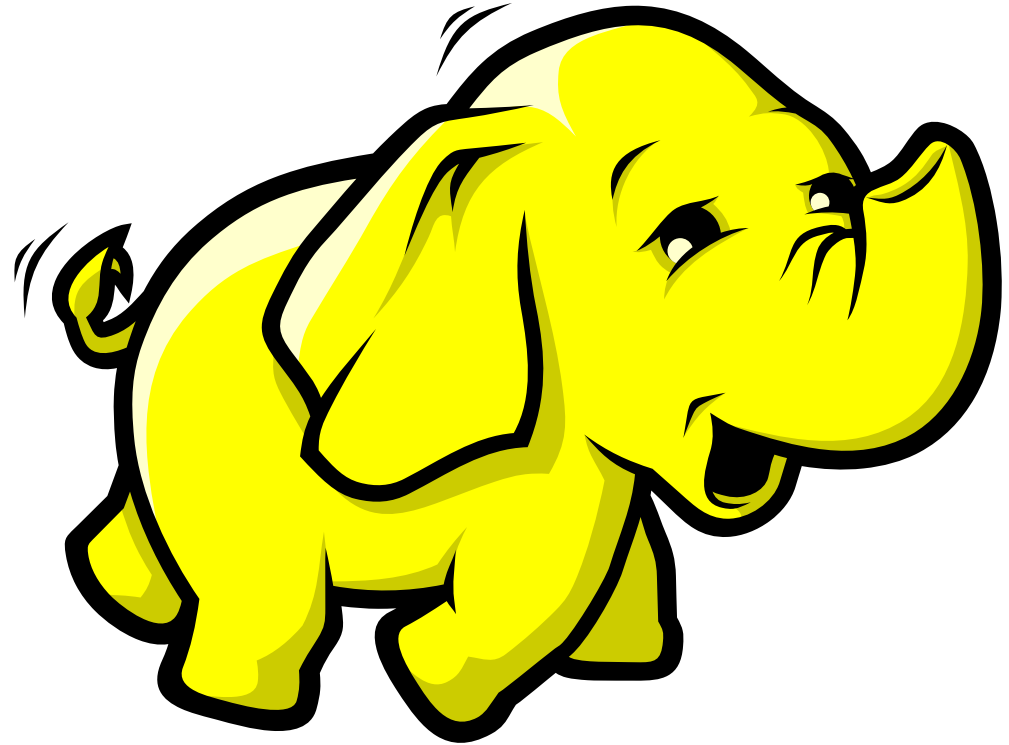
Введение в API

Оптимизации Spark

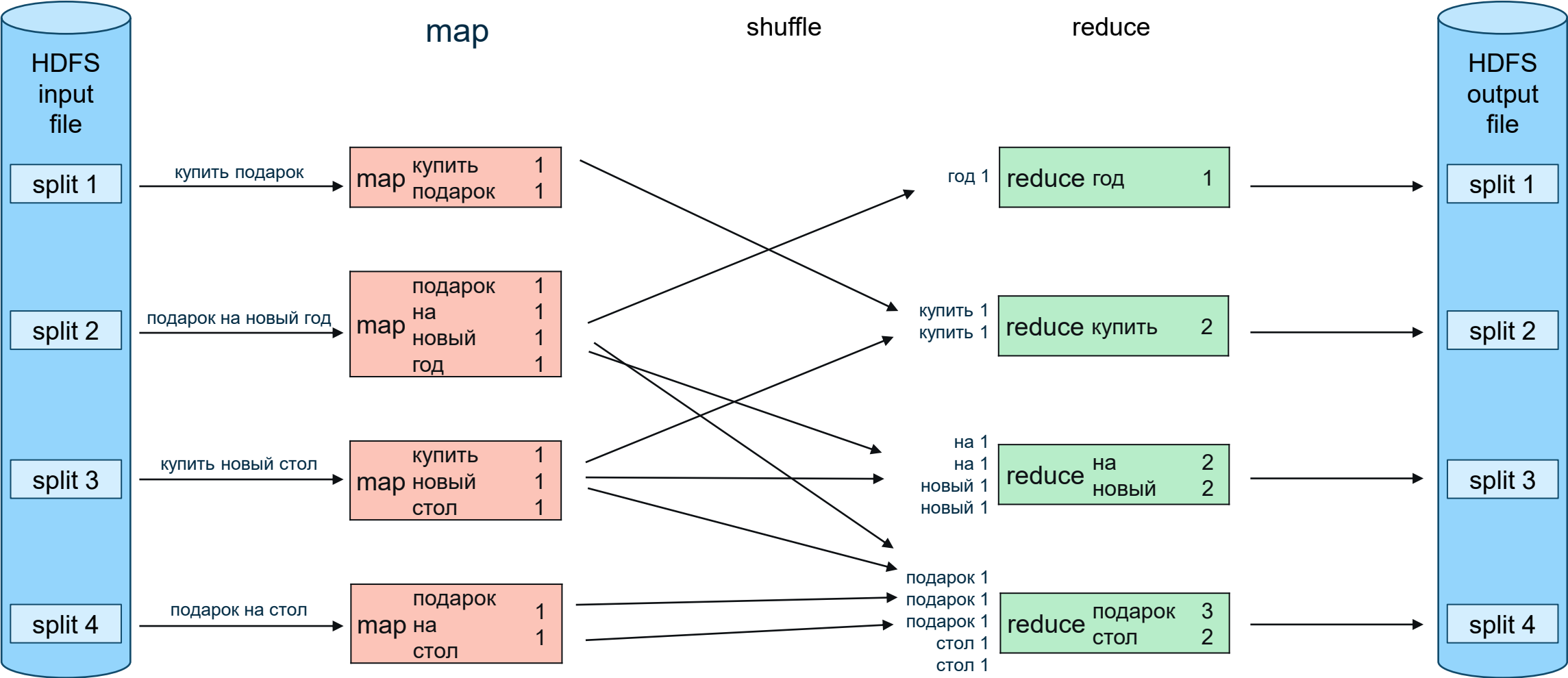


Вспоминаем HDFS + MapReduce

1. HDFS - распределенная файловая система
2. MapReduce - фреймворк распределенных вычислений
3. Масштабируется линейно с увеличением числа узлов
4. Обеспечивает локальность данных



MapReduce

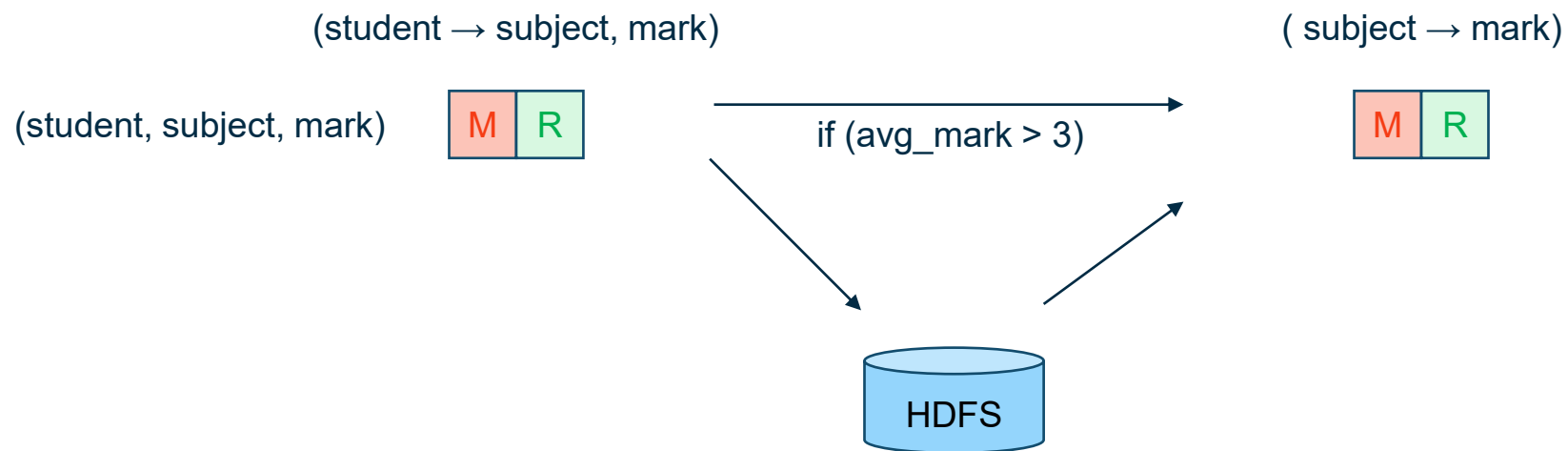


Недостатки Hadoop

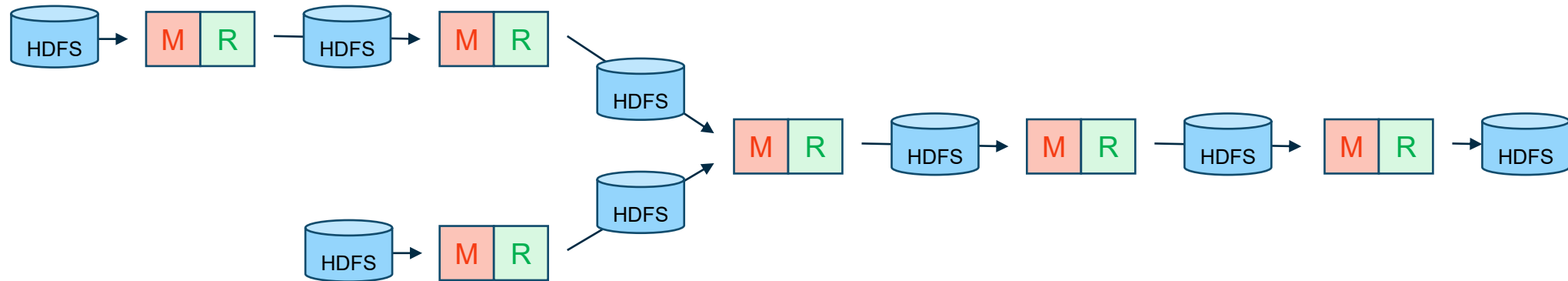
- Любое вычисление требует написания шаблонного кода
- Ориентирован на batch processing
- Сложная логика из нескольких MR джоб требует управления и работает медленно

Пример

Найти среднюю оценку по каждому предмету для студентов, у которых средний балл по всем предметам выше 3.



Большой overhead на IO



Apache Spark

- + Хранение промежуточных данных в памяти
- + Сам строит DAG и выполняет независимые задачи параллельно
- + Простой API (SQL, Scala, Python, Java, R)
- + Модульность (Structured Streaming, MLlib, GraphX)
- + Расширяемость



Демо

synthetic-employee-dataset.json

Основные концепции Spark

Application

Driver

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, regexp_replace
```

```
spark = SparkSession.builder \
    .appName("PySpark Sample Code") \
    .getOrCreate()
```

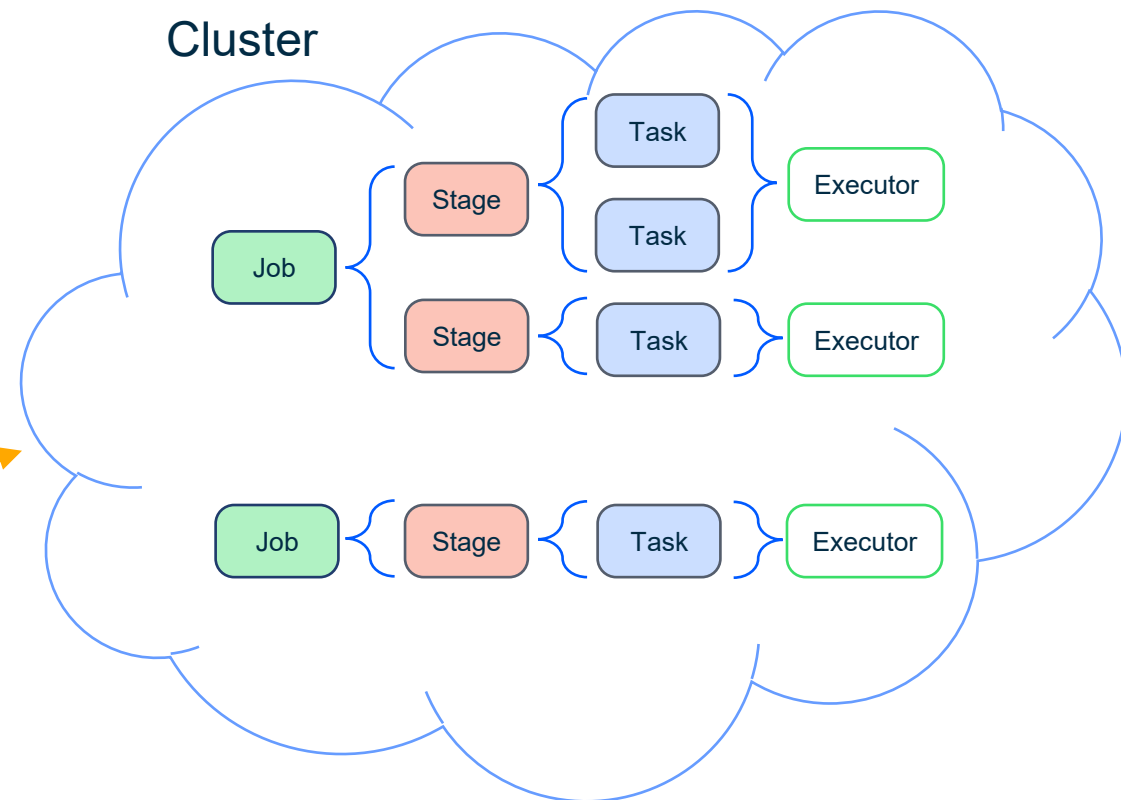
```
df_filtered = df.filter(col("age") > 10)
df_filtered.show()
```

```
....
```

```
df_selected = df.select("name", "age")
df_selected.show()
```

SparkSession

Cluster



Основные концепции Spark

Application – пользовательская программа, состоит из драйвера и ассоциированных ресурсов на стороне кластера

Драйвер – процесс управляющий исполнением программы через SparkSession

SparkSession – объект Spark API через который запускается функциональность Spark

Job – вычисление, состоящее из нескольких тасков

Stage – подмножество тасков (подробнее далее)

Task – минимальная единица исполнения поступающая на выполнение executor

Примеры драйвера

- Интерактивная сессия Jupyter notebook или ipython
- Scala Shell (spark-shell) в терминале
- Программа написанная на языке Java

Демо

Введение в Spark API

- Spark SQL
- Pandas API on Spark
- Structured Streaming
- MLlib

Введение в Spark API

- Данные представляются в Spark в виде объектов класса Dataset (DataFrame для Python)
- Операции, которые можно применить к DataFrame делятся на **трансформации** и **действия**
- Применение **трансформации** к DataFrame возвращает другой DataFrame. DataFrame неизменяем
- **Трансформации** исполняются лениво
- **Действия** запускают вычисления на кластере и все отложенные **трансформации**

Примеры операций Spark

Трансформации

`filter(condition)`

Фильтрует строки по условию

```
old_aged = df.filter(df.age > 3)
```

`select(*cols)`

Выбрать колонки

```
age_and_size_df = df.select(df.age, "size")
```

`sort(*cols)`

Сортировать по колонке

```
size_sorted_df = df.sort("size")
```

`join(*cols)`

Производит слияние

```
df.join(files_df, "author")
```

Примеры операций Spark

Действия

`count()`

Возвращает число строк

```
print(df.filter(df.age > 3).count())
```

`show(n=20)`

Выводит часть строк

```
df.select(df.age, "size").show()
```

`take(num)`

Возвращает num первых строк в виде списка

```
oldest= df.sort("age",  
ascending=False).take(1)
```

`write.json(path)`

Записывает данные в кластер

```
df.write.json("/data/queries/2026-01-01")
```

Демо

Почему операции
делятся на **ленивые**
трансформации и
действия?



Оптимизации

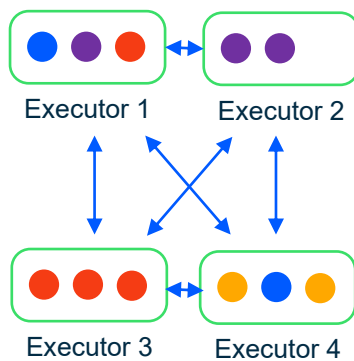
- Данные в DataFrame разделены на разделы (partitions) и распределены
- Spark стремится минимизировать IO, например, назначает задачи на ноды с данными

Оптимизации

```
purchases_df.join(users_df, "user_id").filter(users_df.city == "Тверь").select("price").collect()
```

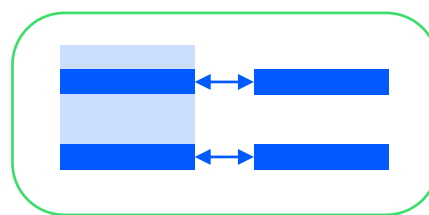
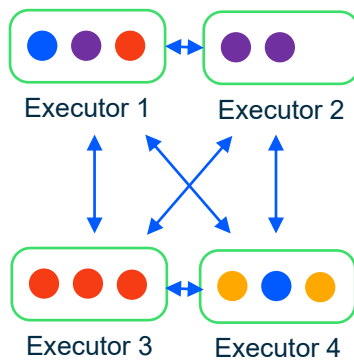
Оптимизации

```
purchases_df.join(users_df, "user_id").filter(users_df.city == "Тверь").select("price").collect()
```

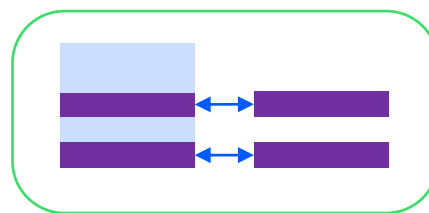


Оптимизации

```
purchases_df.join(users_df, "user_id").filter(users_df.city == "Тверь").select("price").collect()
```



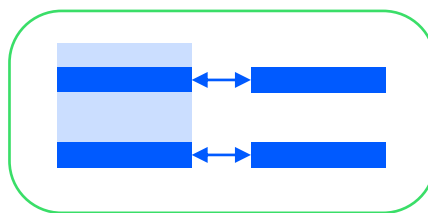
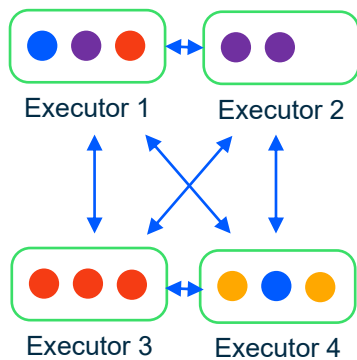
Executor 1



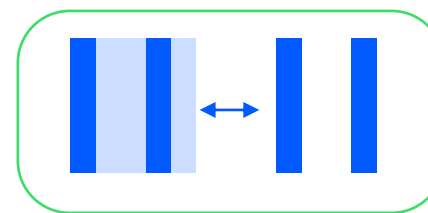
Executor 2

Оптимизации

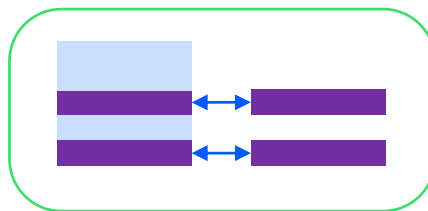
```
purchases_df.join(users_df, "user_id").filter(users_df.city == "Тверь").select("price").collect()
```



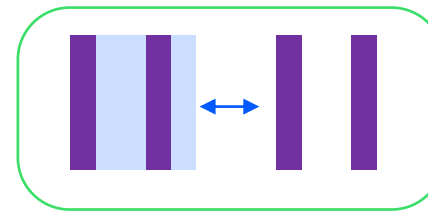
Executor 1



Executor 1



Executor 2



Executor 2

Узкие и широкие трансформации

- Узкие трансформации для расчета требуют один входной раздел данных и один выходной. То есть для исполнения не требуют обмена данными из других нод
- Широкие трансформации требуют данных от разных разделов данных, следовательно требуют обмена данными. Такие операции задействуют shuffle
- Job – набор задач (tasks) для выполнения действия (action)
- Stage – последовательность трансформаций, которые не требуют обмена данными между разделами. Stage разделяются обменом (shuffle)

Типы Join

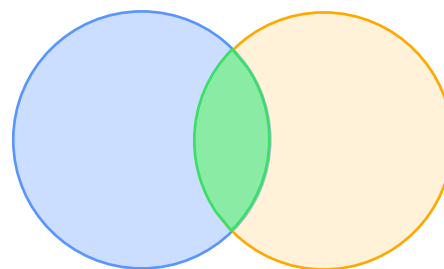
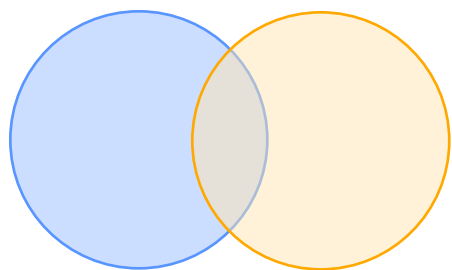
Типы Join

inner

actor_id	name
1	Леонардо ДиКаприо
2	Кейт Уинслет
3	Брэд Питт
5	Том Хэнкс

actor_id	movie_title
1	Титаник
2	Титаник
4	Джокер
6	Гладиатор

actor_id	name	movie_title
1	Леонардо ДиКаприо	Титаник
2	Кейт Уинслет	Титаник



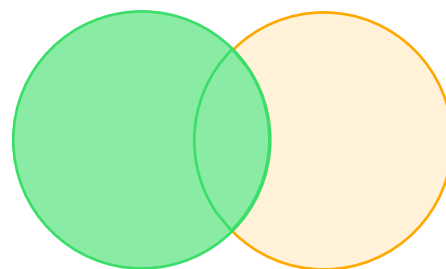
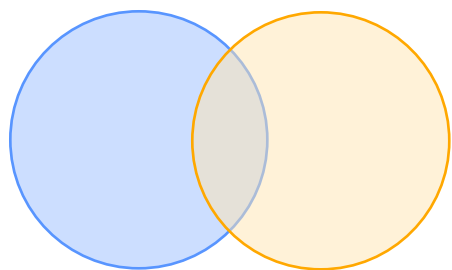
Типы Join

left

actor_id	name
1	Леонардо ДиКаприо
2	Кейт Уинслет
3	Брэд Питт
5	Том Хэнкс

actor_id	movie_title
1	Титаник
2	Титаник
4	Джокер
6	Гладиатор

actor_id	name	movie_title
1	Леонардо ДиКаприо	Титаник
2	Кейт Уинслет	Титаник
3	Брэд Питт	NULL
5	Том Хэнкс	NULL



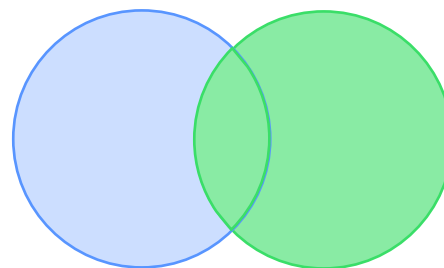
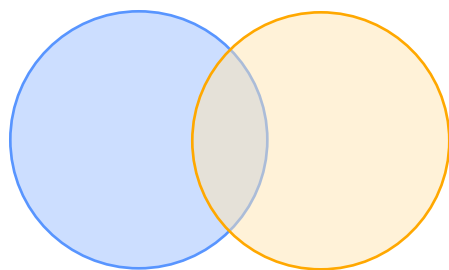
Типы Join

right

actor_id	name
1	Леонардо ДиКаприо
2	Кейт Уинслет
3	Брэд Питт
5	Том Хэнкс

actor_id	movie_title
1	Титаник
2	Титаник
4	Джокер
6	Гладиатор

actor_id	name	movie_title
1	Леонардо ДиКаприо	Титаник
2	Кейт Уинслет	Титаник
4	NULL	Джокер
6	NULL	Гладиатор

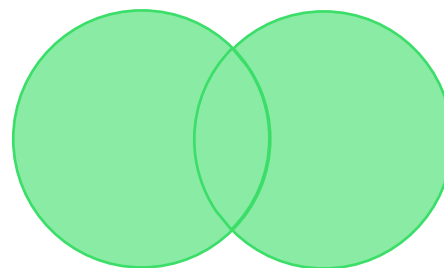
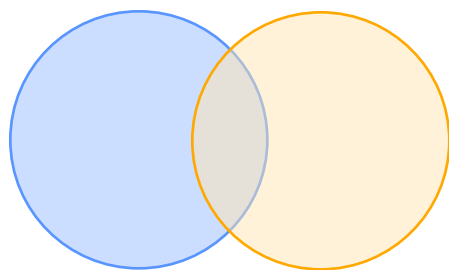


Типы Join full

actor_id	name
1	Леонардо ДиКаприо
2	Кейт Уинслет
3	Брэд Питт
5	Том Хэнкс

actor_id	movie_title
1	Титаник
2	Титаник
4	Джокер
6	Гладиатор

actor_id	name	movie_title
1	Леонардо ДиКаприо	Титаник
2	Кейт Уинслет	Титаник
3	Брэд Питт	NULL
4	NULL	Джокер
5	Том Хэнкс	NULL
6	NULL	Гладиатор



Типы Join

cross

actor_id	name
1	Леонардо ДиКаприо
2	Кейт Уинслет
3	Брэд Питт
5	Том Хэнкс

actor_id	movie_title
1	Титаник
2	Титаник
4	Джокер
6	Гладиатор

actor_id	name	movie_title
1	Леонардо ДиКаприо	Титаник
1	Леонардо ДиКаприо	Титаник
1	Леонардо ДиКаприо	Джокер
1	Леонардо ДиКаприо	Гладиатор
2	Кейт Уинслет	Титаник
	...	
2	Кейт Уинслет	Гладиатор
	...	
5	Том Хэнкс	Титаник
	...	
5	Том Хэнкс	Гладиатор

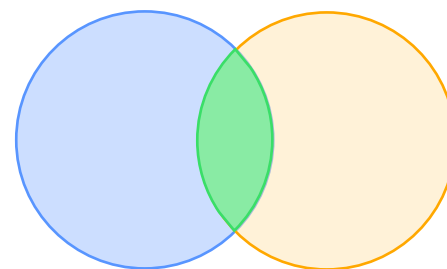
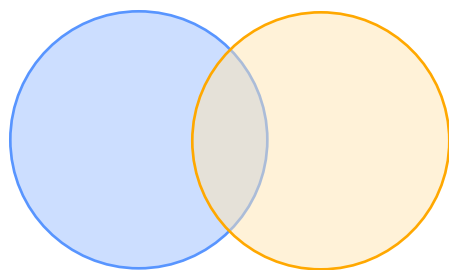


Типы Join semi

actor_id	name
1	Леонардо ДиКаприо
2	Кейт Уинслет
3	Брэд Питт
5	Том Хэнкс

actor_id	movie_title
1	Титаник
2	Титаник
4	Джокер
6	Гладиатор

actor_id	name
1	Леонардо ДиКаприо
2	Кейт Уинслет



*Только данные
из левой таблицы

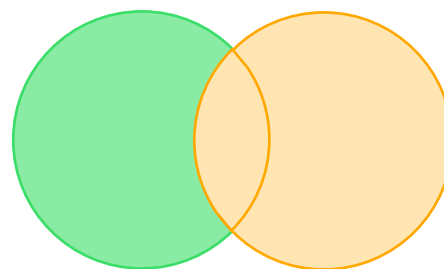
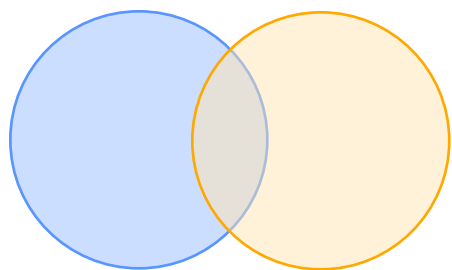
Типы Join

anti

actor_id	name
1	Леонардо ДиКаприо
2	Кейт Уинслет
3	Брэд Питт
5	Том Хэнкс

actor_id	movie_title
1	Титаник
2	Титаник
4	Джокер
6	Гладиатор

actor_id	name
3	Брэд Питт
5	Том Хэнкс



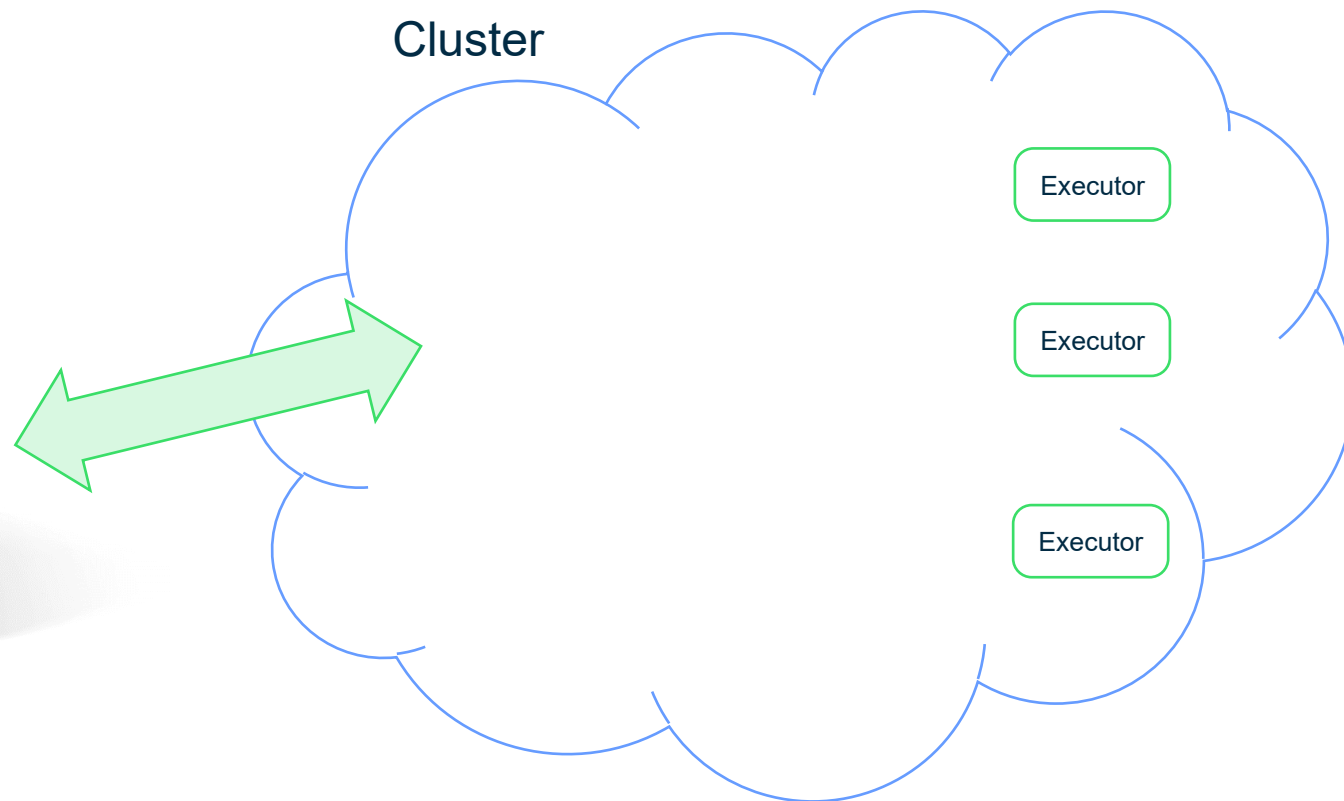
*Только данные
из левой таблицы

Deploy client mode

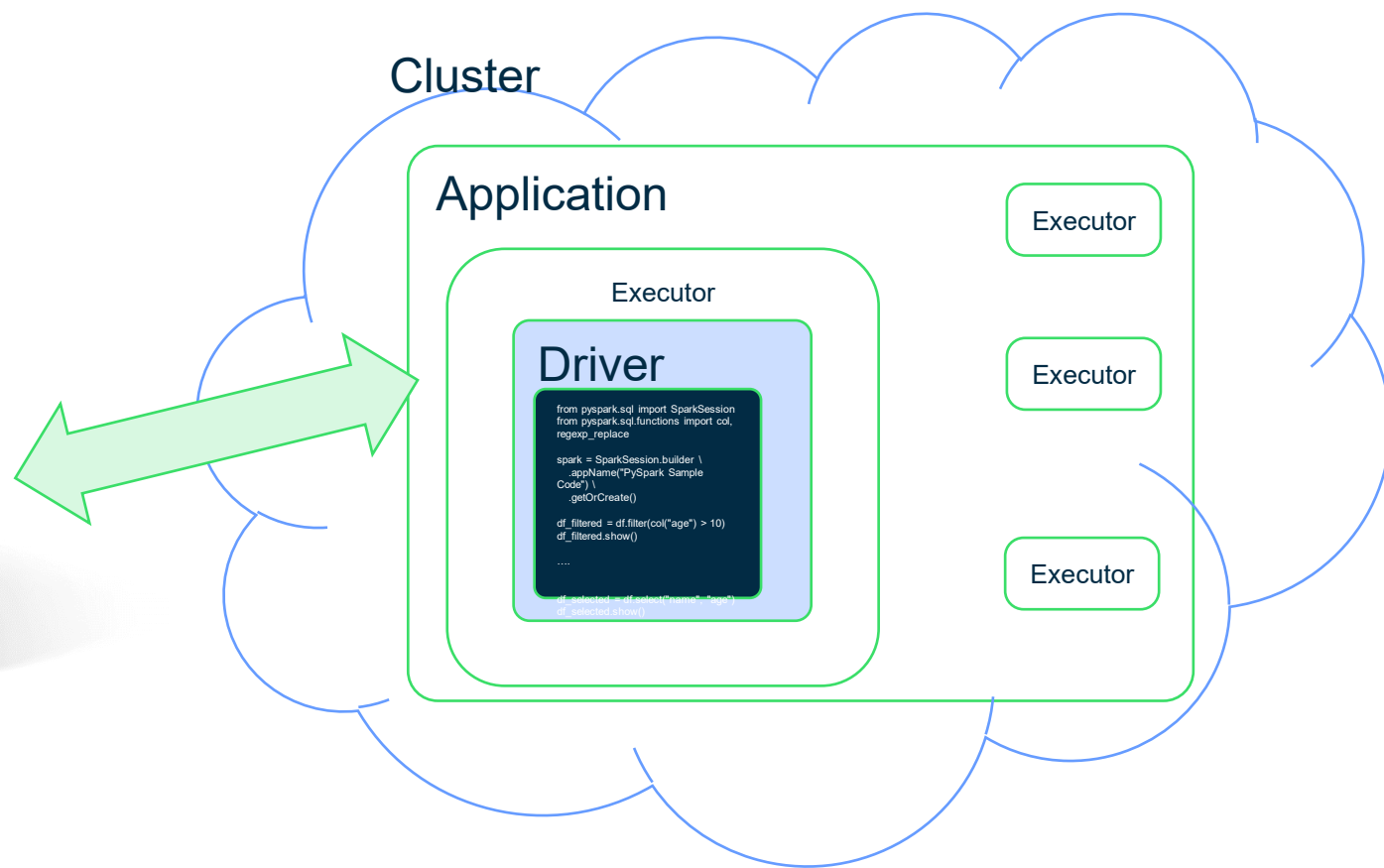
Application



Cluster



Deploy cluster mode



Минусы Apache Spark

- Данные должны помещаться в память!
- Менее надежен чем hadoop
- Дороже в эксплуатации

Подведем итоги

- Apache Spark – фреймворк распределенных вычислений
- Spark нацелен на скорость, для ускорения вычислений он применяет оптимизации
- Для возможности оптимизации операции над данными делятся на трансформации и действия
- Трансформации лениво откладываются, действия запускают исполнение запроса (job)
- Трансформации бывают узкие (не требующие обмена данными) и широкие (требующие shuffle)
- Все задачи (tasks) разбиваются по этапам (stage) по границам shuffle
- Скорость Spark обеспечивается памятью кластера